

INFORME TÉCNICO DE COMPETITIVIDAD Y ESTRATEGIA DE TAKEOUT

GOOGLE BIGQUERY VS. DATABRICKS

Preparado para: Equipo Técnico de Arquitectura y Ventas - Caso Especial: ADEVINTA
Fecha: Julio 2026 | Clasificación: Confidencial / Uso Interno

RESUMEN EJECUTIVO

Adevinta, como líder global en marketplaces digitales y clasificados (operando marcas de alta transaccionalidad como Fotocasa, InfoJobs, Coches.net, Habitacalia y leboncoin.fr), gestiona volúmenes masivos de datos con requisitos críticos de baja latencia, democratización de BI y escalabilidad para casos de uso de Inteligencia Artificial. Actualmente, la presencia de Databricks introduce una complejidad operativa y un sobre coste significativo debido a su arquitectura basada en clusters (Spark) y el licenciamiento doble (DBU + Infraestructura de Cloud).

Este informe proporciona al equipo técnico de Google Cloud la base argumental, las diferencias arquitectónicas profundas, los esquemas de gobernanza y una estrategia de migración paso a paso ("Takeout") para posicionar a Google BigQuery como la plataforma unificada de datos e IA en Adevinta, reemplazando de forma eficiente y con menor TCO la arquitectura de Databricks.

1. COMPARATIVA ARQUITECTÓNICA PROFUNDA

La principal vulnerabilidad de Databricks reside en su herencia tecnológica. Aunque se comercializa como un "Data Intelligence Platform", Databricks sigue dependiendo fundamentalmente de un motor de ejecución Apache Spark autogestionado sobre máquinas virtuales de nube. Esto introduce fricción técnica y operativa en comparación con el diseño puramente serverless y desagregado de BigQuery.

1.1 BigQuery: Arquitectura Desagregada Serverless (Dremel + Colossus)

BigQuery utiliza una arquitectura completamente desagregada donde el almacenamiento y el cómputo se escalan de forma independiente y automática a través de una red interna de ultra-alta velocidad (Jupiter).

- **Cómputo (Dremel):** Un motor de ejecución de consultas SQL distribuido y multi-inquilino que utiliza un diseño de árbol de ejecución. BigQuery asigna dinámicamente miles de "slots" (unidades de procesamiento) según la complejidad de la consulta.
- **Almacenamiento (Colossus):** El sistema de archivos distribuido de Google que almacena los datos en formato columnar estructurado (Capacitor), optimizando la compresión y la velocidad de escaneo.
- **Interconexión (Jupiter Network):** Permite mover petabytes de datos entre Colossus y Dremel a una velocidad de 1 Terabit por segundo por rack, eliminando la necesidad de mantener datos localmente en los nodos de ejecución.

1.2 Databricks: Arquitectura de Control y Plan de Datos Basado en VM

En Databricks, las cargas de trabajo se ejecutan en clusters tradicionales de máquinas virtuales que el cliente o el plano de control deben aprovisionar, configurar y optimizar.

- Plan de Control (Control Plane): Gestionado por Databricks, aloja la interfaz de usuario, los cuadernos (notebooks) y la orquestación de jobs.
- Plan de Datos (Data Plane): Reside en la suscripción de nube del cliente. Requiere el despliegue de clusters de cómputo (e.g., VMs de AWS o GCP como GKE) que leen de buckets de almacenamiento de objetos (Amazon S3 o GCS).

Diagrama de Arquitecturas Comparadas

```
=====
ARQUITECTURA BIGQUERY (SERVERLESS)
=====
[ Clientes: BI, SQL, Vertex AI, BigQuery Studio ] (Acceso único vía API/Consola)
|
[ Capa de Cómputo Distribuido (Dremel) ] <-- Auto-escalado instantáneo sin clusters
|
[ Jupiter Network (1 Tbps+) ]
|
[ Capa de Almacenamiento Global (Colossus) ] <-- Formatos nativos e híbridos (Iceberg)
=====

=====
ARQUITECTURA DATABRICKS (BASADA EN CLUSTERS)
=====
[ Plano de Control (Databricks SaaS Portal, Notebooks, Unity Catalog) ]
| (Orquestación y envío de tareas)
v
[ Plano de Datos (Suscripción Cliente) ]
+-----+
| [ Cluster Spark / SQL Warehouse ] (VMs Activas: Nodos Driver + Workers) |
| * Requiere arranque en frío (Cold Start: ~3-5 min) |
| * Requiere configuración de auto-escalado y tamaño de máquina |
+-----+
| (Lectura de datos vía Red / Egress)
v
[ Almacenamiento de Objetos de Nube ] (GCS / S3 / Azure Blob - Delta/Parquet)
=====
```

1.3 El Impacto del "Operational Toil" (Esfuerzo Operativo) en Adevinta

Para el equipo técnico de Adevinta, operar Databricks se traduce en tareas constantes de bajo nivel que restan productividad:

1. Ajuste y Sizing de Clusters: Decidir el tipo de máquina virtual (memoria optimizada, cómputo optimizado), configurar el número mínimo/máximo de nodos y parametrizar Spark.
2. Cold Starts (Tiempos de Arranque en Frío): Un SQL Warehouse o un cluster de notebooks inactivo tarda entre 2 y 7 minutos en provisionar la infraestructura subyacente. En BigQuery, el arranque de cualquier consulta es sub-segundo.
3. Mantenimiento de Tablas: Necesidad de ejecutar comandos manuales como VACUUM u OPTIMIZE de manera periódica en Delta Lake para evitar la degradación del rendimiento de lectura. BigQuery automatiza la optimización física, la compactación de metadatos y la ordenación de datos en segundo plano sin coste ni intervención del usuario.

2. ESTRATEGIA DE OPEN LAKEHOUSE E INTEROPERABILIDAD

Databricks se posiciona fuertemente como el creador del concepto "Lakehouse" a través de Delta Lake. Sin embargo, su gobernanza ejerce un fuerte bloqueo de proveedor (vendor lock-in), especialmente con Delta format, mientras que el mercado y la industria se están estandarizando masivamente hacia Apache Iceberg.

2.1 BigQuery Managed Tables for Apache Iceberg

Google BigQuery ofrece soporte nativo y de primer nivel para Apache Iceberg, proporcionando una experiencia de Lakehouse abierto sin precedentes:

- Almacenamiento Unificado de Copia Única: BigQuery puede consultar y escribir directamente en tablas formateadas en Apache Iceberg en almacenamiento de objetos externo (GCS, S3, ADLS) sin mover los datos.
- BigLake: La capa de abstracción de almacenamiento de Google que permite a BigQuery aplicar controles de seguridad detallados (a nivel de fila y columna) sobre datos almacenados en formatos abiertos (Parquet, Avro, Orc, Delta, Iceberg).
- Rendimiento Equivalente a Tablas Nativas: Con las tablas gestionadas de BigQuery para Iceberg, el rendimiento de consulta es equivalente a las tablas nativas gracias al motor BI Engine y la aceleración de metadatos de BigQuery.

2.2 Desarmando el argumento de "UniForm" de Databricks

Databricks promueve Universal Format (UniForm) para permitir que Delta Lake sea leído como Iceberg. Sin embargo, esta traducción introduce:

1. Latencia de Generación de Metadatos: La generación de metadatos Iceberg se realiza de forma asíncrona, lo que puede provocar que los lectores externos vean datos desactualizados en comparación con los lectores nativos de Delta.
2. Complejidad Operativa: El equipo de Adevinta debe configurar e instrumentar Unity Catalog para habilitar UniForm en cada tabla de forma individual, aumentando el riesgo de errores de configuración.

3. SEGURIDAD, GOBERNANZA Y CATÁLOGO UNIFICADO

Para una corporación como Adevinta, con múltiples unidades de negocio autónomas y equipos descentralizados, la seguridad y la gobernanza de datos son prioridades absolutas.

3.1 Dataplex (Universal Catalog) vs. Databricks Unity Catalog

Característica	Google Dataplex (Universal Catalog)	Databricks Unity Catalog
Gobernanza Multicloud	Nativa y consistente en GCP, AWS y Azure a través de BigQuery Omni y Dataplex.	Limitada principalmente a la plataforma Databricks; carece de integración profunda con servicios nativos de otros proveedores.
Políticas de Seguridad	Seguridad unificada integrada con Cloud IAM. Soporta políticas a nivel de columna, fila y enmascaramiento dinámico de datos.	Modelo de seguridad aislado de la IAM nativa del Cloud. Requiere duplicar y sincronizar permisos en Unity Catalog.
Linaje de Datos (Data Lineage)	Automatizado y de extremo a extremo, desde la ingestión hasta Looker y Vertex AI, capturando automáticamente el historial de consultas de BigQuery.	Captura linaje principalmente dentro de las tareas ejecutadas en Databricks; visibilidad muy limitada de herramientas externas sin SDKs complejos.
Gobernanza de IA	Registro, evaluación y gobierno de modelos integrados de forma nativa a través de la suite de Vertex AI.	Requiere integrar MLflow con Unity Catalog, añadiendo otra capa de abstracción.

3.2 El "Impuesto de Integración" de Unity Catalog

Unity Catalog actúa como un silo de gobernanza secundario. Si Adevinta desea utilizar servicios nativos de la nube (como Vertex AI, AWS Bedrock o herramientas de BI externas), se ve obligada a pasar por las APIs de Databricks o a duplicar el modelo de seguridad. Con BigQuery y Dataplex, la gobernanza se aplica una sola vez a nivel de almacenamiento y cómputo de la nube, protegiendo todos los accesos de analistas, científicos de datos y agentes de IA de forma consistente.

4. CAPACIDADES DE AI NATIVAS Y COMPUTACIÓN MULTIMODAL

Databricks requiere que los científicos de datos escriban y despliegan código en cuadernos Spark, lo que limita la democratización de la Inteligencia Artificial al excluir a los analistas de negocio y de datos que dominan el lenguaje SQL.

4.1 BigQuery ML (BQML): Democratización de la IA en Adevinta

BigQuery ML permite a los analistas de Adevinta construir, evaluar y ejecutar modelos de Machine Learning directamente en BigQuery utilizando SQL estándar, eliminando la necesidad de exportar datos a entornos Spark externos.

- Modelos Soportados: Regresión lineal/logística, XGBoost, K-means, series temporales (ARIMA+), y redes neuronales profundas (TensorFlow).
- Integración Vertex AI: BQML se conecta nativamente con Vertex AI para registrar modelos automáticamente en el Vertex Model Registry o invocar LLMs como Gemini mediante funciones SQL simples (ML.GENERATE_TEXT).

4.2 BigQuery Studio: Un Único Workspace para Todos

A diferencia del entorno notebook-céntrico de Databricks que margina a los perfiles SQL, BigQuery Studio proporciona una interfaz colaborativa unificada:

- Políglota: Permite alternar sin problemas entre celdas SQL, Python y Javascript dentro de la misma interfaz de edición.
- Integración de Git y CI/CD: Conexión nativa con repositorios para control de versiones de código y despliegue automatizado.
- Asistencia por IA: Incorporación profunda de Gemini Code Assist para la generación de código, depuración y documentación automática de consultas y modelos de datos.

5. ANÁLISIS DE TCO Y EFICIENCIA OPERATIVA

Databricks introduce costes significativos y a menudo ocultos debido a su modelo de tarificación fragmentado.

5.1 Los Tres Factores de Coste Ocultos en Databricks

1. Doble Facturación (Double-Billing): Adevinta debe pagar a Databricks por las unidades de procesamiento de Databricks (DBUs) consumidas, además de pagar al proveedor de cloud (e.g., AWS o GCP) por las máquinas virtuales (VMs) de cómputo del cluster que ejecutan dichas DBUs.
2. Cómputo Inactivo (Idle Clusters): Los clusters autogestionados tardan en escalar hacia abajo. Si un cluster se configura para apagarse tras 20 minutos de inactividad, Adevinta sigue pagando tanto las DBUs como las VMs por ese periodo de inactividad. En BigQuery, el cómputo se apaga instantáneamente tras la ejecución de la consulta.
3. Egress y Tránsito de Red: El movimiento de datos entre buckets de almacenamiento y el Data Plane de Databricks (cuando están en diferentes zonas de disponibilidad o VPCs) genera costes de transferencia de red invisibles pero masivos a gran escala.

5.2 Ventajas Financieras de BigQuery

- Serverless Real: Cero coste por inactividad. Solo se factura por los datos escaneados (modelo On-Demand) o mediante reservas de capacidad física (Slots / Editions) donde se paga exactamente por la potencia asignada con compromisos predecibles.
- Compresión de Almacenamiento: BigQuery Capacitor comprime los datos de forma masiva (frecuentemente reduciendo el espacio físico entre un 3x y un 5x), reduciendo los costes de almacenamiento base comparado con Delta Lake tradicional.
- Casos de Éxito de Migración:
 - JB Hunt: Logró un ahorro del 60% en TCO tras migrar de Databricks a BigQuery mediante la desclasificación de plataformas heredadas y el uso de recursos serverless eficientes.
 - Yassir: Migró de Databricks a BigQuery unificando su almacén de datos y Vertex AI, eliminando silos de cómputo y optimizando costes de entrenamiento de modelos ML.

6. PLAN DE MIGRACIÓN Y TAKEOUT PARA ADEVINTA

Para afrontar con éxito un proceso de Databricks Takeout en Adevinta, el equipo técnico debe seguir una estrategia estructurada que minimice el riesgo de interrupción y demuestre valor rápido.

```
=====
ESTRATEGIA DE TAKEOUT EN 4 FASES PARA ADEVINTA
=====
[ Fase 1: Descubrimiento y Evaluación ]
* Clasificar cargas de trabajo: ¿Es SQL directo o Spark complejo?
* Mapear catálogos en Unity Catalog hacia Dataplex.
|
v
[ Fase 2: Modernización del Almacenamiento (Cero Movimiento de Datos) ]
* Configurar BigLake sobre el almacenamiento actual (S3/GCS).
* Habilitar BigQuery Managed Tables para Apache Iceberg.
|
v
[ Fase 3: Migración de Pipelines y Cómputo ]
* SQL Warehouses de Databricks --> BigQuery Editions (Reservas de Cómputo).
* Notebooks Spark complejos --> Dataproc Serverless o Python en BQ Studio.
|
v
[ Fase 4: Integración AI y Activación (Looker + Vertex AI) ]
* MLflow/Mosaic AI --> Vertex AI Model Registry y BigQuery ML.
* Dashboards/Analítica --> Conexión nativa con Looker (BI Engine).
=====
```

6.1 Guía de Sustitución Tecnológica Directa (Takeout Battlecard)

Componente Databricks	Alternativa Objetivo en GCP	Estrategia de Sustitución
Databricks SQL Warehouses	BigQuery (Editions / Slots)	Migración directa de sintaxis SQL. BigQuery se encarga del auto-escalado instantáneo sin necesidad de aprovisionar tamaños de warehouse.
Notebooks Spark (Ingesta/ETL)	Dataproc Serverless / Cloud Dataflow	Para pipelines Spark de alta complejidad que no puedan convertirse a SQL, desplegar Dataproc Serverless. Esto elimina la gestión de clusters Spark.
Unity Catalog	Dataplex (Universal Catalog) + IAM	Consolidar la gobernanza de datos y metadatos en un único plano de control unificado a nivel de nube, eliminando el silo secundario.
MLflow & Mosaic AI	Vertex AI Platform + BQML	Migrar los experimentos y el registro de modelos a Vertex AI Model Registry. Utilizar BigQuery ML para el entrenamiento y despliegue con SQL.
Delta Lake Formats	Apache Iceberg / Tablas Nativas BQ	Configurar BigLake para leer las tablas Delta existentes o, idealmente, migrar el almacenamiento de manera automatizada a formato Apache Iceberg.

CONCLUSIÓN Y SIGUIENTES PASOS

El posicionamiento competitivo frente a Databricks en Adevinta no debe basarse únicamente en una guerra de rendimiento sintético, sino en la simplicidad operativa (verdadero Serverless), la interoperabilidad abierta (Apache Iceberg con BigLake) y un TCO radicalmente inferior.

Para iniciar el proceso de Takeout con los técnicos de Adevinta, se proponen los siguientes pasos inmediatos:

1. Taller de Descubrimiento de TCO: Analizar los costes reales de Databricks de Adevinta (sumando DBU + coste de VMs de nube infrautilizadas) para estructurar el caso de negocio financiero.
2. Prueba de Concepto (POC) con BigLake e Iceberg: Seleccionar un pipeline de datos de alto volumen actualmente gobernado en Databricks e implementar una lectura/escritura serverless en BigQuery utilizando formatos abiertos de Iceberg, demostrando el fin de la duplicación de datos y la eliminación de la administración de clusters.
3. Habilitación de BigQuery Studio: Introducir a los científicos de datos de Adevinta en BigQuery Studio para demostrar cómo pueden seguir programando en Python sin sufrir el "Operational Toil" de administrar la infraestructura Spark subyacente.